

BILL OF MATERIALS

Name	Description	Quantity
Raspberry Pi Zero W/Raspberry Pi 4/ Raspberry Pi 5	Single-board computer (SBC)	1
RPi camera	4MP Raspberry Pi CSI camera	1
Voice bonnet/Sound HAT	Raspberry Pi audio HAT	1
4Ω-10Ω speaker	Speaker for Raspberry Pi	2
Raspberry Pi CSI camera cable	CSI FPC cable	1
5V, 2A AC-DC adaptor	5V, 2A power supply for Raspberry Pi and speaker	1

```
import openwakeword
from openwakeword.model import Model
import pyaudio
import speech_recognition as sr
from openai import OpenAI
import numpy as np
from gtts import gTTS
import pygame
import os
import time

# ===== CONFIG =====
WAKE_WORD = "alexa" # Try "alexa", "hey mycroft", "hey siri", etc.
AI_NAME = "alexa"

PROVIDER = "gemini" # Options: "gemini", "openai", "grok", "deepseek"

# ===== Initialize Pygame =====
pygame.mixer.pre_init(44100, -16, 2, 2048)
pygame.mixer.init()

# ===== API CLIENT =====
def create_client():
    if PROVIDER == "gemini":
```

Fig. 4: Importing a Python module and configuring the AI name and wake word

```
pygame.mixer.pre_init(44100, -16, 2, 2048)
pygame.mixer.init()

# ===== API CLIENT =====
def create_client():
    if PROVIDER == "gemini":
        return OpenAI(
            api_key="AIzaSyCXI3J6mu88vn5E-GwKCus9FaM280TP44Q",
            base_url="https://generativelanguage.googleapis.com/v1beta/openai/"
        )
    # Add other providers if needed...

client = create_client()

# ===== IMPROVED SPEAK FUNCTION =====
def speak(text):
    print(f"[AI_NAME]: {text}")
    try:
        tts = gTTS(text=text, lang='en', tld='com.au') # Best natural female tone
        filename = "temp_response.mp3"
        tts.save(filename)

        pygame.mixer.music.load(filename)
        pygame.mixer.music.play()

        # Wait until speech is completely finished
        while pygame.mixer.music.get_busy():
            time.sleep(0.1)

        pygame.mixer.music.unload()
        os.remove(filename) # Clean up
    except Exception as e:
        print("TTS Error:", e)

# ===== SIMPLE COMMANDS =====
def handle_simple_command(command):
    cmd = command.lower()
    if any(word in cmd for word in ["light", "lights"]):
        if "on" in cmd:
            speak("Lights turned on")
```

Fig. 5: Code snippet for configuring the API connection to the generative AI model

within view. It can also function as a real-time language interpreter and an assistive device for visually impaired users. The prototype is shown in Fig. 1.

The core of this design is a Raspberry Pi board (such as the Raspberry Pi Zero W, Raspberry Pi 4, Raspberry Pi 5, or a similar model) equipped with Bluetooth Low Energy (BLE) and Wi-Fi connectivity. For audio input and output, a Raspberry Pi-compatible voice HAT with microphone and speaker support is required. In the prototype, a Google Voice Bonnet is used for audio processing. Video capture is handled by the official Raspberry Pi camera, which connects directly to the Raspberry Pi through its CSI camera interface.

Along with these key modules, several supporting components are required to complete the system. The components needed to build the system are listed in the Bill of Materials table.

Software

First, install Raspberry Pi OS and Python on the Raspberry Pi. If the Raspberry Pi already includes a pre-configured SD card, this step can be skipped. The prototype described here uses a Voice Bonnet kit supplied with Raspberry Pi OS, Python, and the required audio configurations pre-installed.

Next, obtain an API key from the generative AI service intended for use. The software supports multiple LLM platforms, including OpenAI ChatGPT, Gemini, Grok, DeepSeek, and others. Simply create an account with the preferred provider and generate an API key. Depending on the provider and model, access may be free or paid.

For this system, Google's free Gemini model is used. After logging into the Gemini API dashboard, generate an API key and save it for later use in the software configuration. Fig. 2 shows the process of creating a Gemini API key.